

Day 18 — SSH Deep Dive

Month 1 • Week 3 • DevOps Training

What Today Is About

Every DevOps action that happens on a remote server happens through SSH. You deploy code over SSH. You restart services over SSH. CI/CD pipelines push to servers over SSH. Ansible runs its playbooks over SSH. Kubernetes nodes are bootstrapped over SSH. If you cannot use SSH confidently and understand what is happening under the hood, you are blocked at every stage of the work.

Most beginners use SSH as a magic command — they type `ssh user@server`, it works (or it doesn't), and they have no idea why. That is a dangerous position to be in. When SSH fails — wrong key, wrong permissions, wrong config, firewall blocking port 22, host key changed — you need to understand exactly why and fix it without guessing.

Today you will understand SSH completely:

- What a key pair is and why it is more secure than a password
- How the authentication handshake works step by step
- How to generate, manage, and use SSH keys
- How to copy your public key to a server
- How to use the SSH config file to manage multiple servers cleanly
- How to run commands on remote servers without an interactive session
- How to copy files over SSH using `scp` and `rsync`
- How SSH tunnelling works and when you need it

By the end of today, SSH will not be magic. It will be a tool you understand and control.

Section 1: What Is SSH and What Problem Does It Solve?

Before SSH existed, engineers used tools like `telnet` and `rsh` to connect to remote servers. These tools sent everything — including your password — as plain text over the network. Anyone with a packet sniffer on the same network could read your password as you typed it.

SSH (Secure Shell) solves this by encrypting everything. Every byte sent between your machine and the server is encrypted. Even if someone intercepts the packets, they see only scrambled data.

But SSH does more than encrypt the connection. It also authenticates — it proves to the server that you are who you say you are. There are two ways to do this:

Password authentication — you type your password. The server checks it. Simple but has problems:

- Passwords can be brute-forced (attacked by trying millions of combinations)
- If the server is compromised, password hashes can be extracted
- You have to type the password every time — makes automation impossible

Key-based authentication — you use a cryptographic key pair. No password is sent over the network. This is what you will use for everything in DevOps.

Section 2: How Key Pairs Work — The Concept

A key pair consists of two mathematically linked keys:

- **Private key** — kept on your machine. Never shared. Never copied to a server. Never emailed. Never put in a git repository. If someone gets your private key, they can impersonate you.
- **Public key** — derived from the private key. Safe to share freely. You copy this to any server you want to connect to. It goes in the server's `~/.ssh/authorized_keys` file.

The magic: **anything encrypted with the public key can only be decrypted with the private key, and vice versa**. They are a matched pair. One without the other is useless.

How authentication works — step by step

When you run `ssh user@server`, here is what happens behind the scenes:

Step 1 – TCP connection

Your machine connects to the server on port 22 (TCP)

Step 2 – Server identification

The server sends its host key fingerprint

Your SSH client checks if this fingerprint matches `~/.ssh/known_hosts`

If it's the first time: "Are you sure you want to connect?" – you say yes

If it changed: warning! Could be a man-in-the-middle attack

Step 3 – Encryption negotiation

Client and server agree on which encryption algorithm to use

A shared session key is negotiated (Diffie-Hellman key exchange)

From this point on, everything is encrypted

Step 4 – Authentication

Server says: "Prove who you are"

Server generates a random challenge and encrypts it with your public key

(The public key is in the server's `authorized_keys` file)

Your SSH client decrypts the challenge using your private key

Your client sends the decrypted result back

Server verifies the result – only someone with the real private key could decrypt it

Authentication succeeds

Step 5 – Shell

Server opens a shell session and sends it to you over the encrypted tunnel

The critical insight: **your private key never leaves your machine**. The server never sees it. The server only has your public key. Authentication happens by you proving you can decrypt something that was encrypted for your public key — which only works if you have the matching private key.

Section 3: Generating SSH Keys — ssh-keygen

```
# Generate a new key pair using the Ed25519 algorithm
$ ssh-keygen -t ed25519 -C "sravan-devops-training"
```

Breaking down the flags:

- `-t ed25519` — use the Ed25519 algorithm. This is the modern standard. It is faster and more secure than the older RSA algorithm.
- `-C "sravan-devops-training"` — a comment added to the public key. This is for your own reference — it helps you identify which key is which when you have many. Use something descriptive: your name, the purpose, the date.

The command asks you two questions:

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/sravan/.ssh/id_ed25519):
```

Press **Enter** to accept the default path. The default `~/.ssh/id_ed25519` is fine for your primary key.

```
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
```

A passphrase encrypts your private key on disk — so even if someone steals the file, they cannot use it without the passphrase. For learning purposes, press **Enter** twice to skip it. In production, you would use a passphrase with `ssh-agent` to avoid typing it repeatedly.

```
Your identification has been saved in /home/sravan/.ssh/id_ed25519
Your public key has been saved in /home/sravan/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256: xK2V9B1mP3qR5tY7uI8oL0aS2dF4gH6jN8kM1nQ3rT5 sravan-devops-training
The key's randomart image is:
+--[ED25519 256]--+
|      .o.      |
|                |
|                |
|                |
|                |
+---+
...

```

Inspect what was created

```
# List the files in your .ssh directory
$ ls -la ~/.ssh/
total 16
drwx----- 2 sravan sravan 4096 Apr 27 10:30 .
drwxr-xr-x  8 sravan sravan 4096 Apr 27 10:30 ..
-rw-----  1 sravan sravan  399 Apr 27 10:30 id_ed25519
-rw-r--r--  1 sravan sravan   94 Apr 27 10:30 id_ed25519.pub
```

Notice the permissions:

- `id_ed25519` — `rw-----` — readable and writable **only by you**. SSH will refuse to use a private key

that is too permissive.

- `id_ed25519.pub` — `rw-r--r--` — readable by everyone. This is fine — it is the public key.

```
# Look at the public key – this is safe to share
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAINq3JkF2mP7xR8tV0bwwY9sH1cL4nO6pQ2gK5jM8dA3u sravan-
devops-training
```

The public key is one line: `algorithm key-data comment`

```
# Look at the private key – NEVER share this
$ cat ~/.ssh/id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAAA...
...
-----END OPENSSH PRIVATE KEY-----
```

The private key is a block of base64-encoded data wrapped in header/footer lines.

Generating keys for different purposes

In real DevOps work you will have multiple keys — one for GitHub, one for production servers, one for staging:

```
# Key for GitHub
$ ssh-keygen -t ed25519 -C "sravan-github" -f ~/.ssh/id_ed25519_github

# Key for production servers
$ ssh-keygen -t ed25519 -C "sravan-prod-2026" -f ~/.ssh/id_ed25519_prod

# Key for a specific client project
$ ssh-keygen -t ed25519 -C "sravan-clientname" -f ~/.ssh/id_ed25519_clientname
```

The `-f` flag specifies the output filename so they do not overwrite each other. You will manage which key is used for which server in the SSH config file (Section 6).

Section 4: Copying Your Public Key to a Server

To log in to a server with key authentication, your public key must be in the server's `~/.ssh/authorized_keys` file.

The automatic way — ssh-copy-id

```
# ssh-copy-id copies your public key to the server automatically
# You will need to authenticate with your password this one time
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub sravan@192.168.1.200

# Output:
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed:
"/home/sravan/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s)
sravan@192.168.1.200's password:
Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'sravan@192.168.1.200'"
and check to make sure that only the key(s) you wanted were added.
```

After this, `ssh sravan@192.168.1.200` will use your key automatically — no password needed.

The manual way — when ssh-copy-id is not available

Sometimes you cannot use `ssh-copy-id` — maybe you only have access to the server console, or the server is freshly provisioned and you have a different way to add the key (cloud provider console, user data script, etc.).

```
# Step 1 – copy your public key content
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAINq3JkF2mP7xR8... sravan-devops-training

# Step 2 – on the server, add it to authorized_keys
# (you need to be on the server already – via password, console, etc.)

# Create .ssh directory if it does not exist
$ mkdir -p ~/.ssh
$ chmod 700 ~/.ssh

# Add the public key
$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAINq3JkF2mP7xR8... sravan-devops-training" >>
~/.ssh/authorized_keys

# Set the correct permissions – SSH is very strict about this
$ chmod 600 ~/.ssh/authorized_keys
```

Why permissions on authorized_keys matter

SSH enforces strict permission rules as a security measure. If the permissions are too loose, SSH assumes the file may have been tampered with and refuses to use it.

```
# Correct permissions – SSH will accept this
-rw----- (600) ~/.ssh/authorized_keys – only owner can read/write
drwx----- (700) ~/.ssh/ – only owner can enter/read/write

# If permissions are wrong, SSH fails silently
# The key exists but SSH ignores it and falls back to password auth
# This is one of the most common SSH debugging issues

# Fix permissions if they are wrong
$ chmod 700 ~/.ssh
$ chmod 600 ~/.ssh/authorized_keys
$ chmod 600 ~/.ssh/id_ed25519
$ chmod 644 ~/.ssh/id_ed25519.pub
```

Section 5: Connecting to a Server with SSH

```
# Basic connection
$ ssh username@hostname_or_ip

# Examples
$ ssh sravan@192.168.1.200
$ ssh ubuntu@ec2-54-123-45-67.compute-1.amazonaws.com
$ ssh root@10.0.1.5
```

The first connection — known_hosts

The very first time you connect to a server, you see this:

```
The authenticity of host '192.168.1.200 (192.168.1.200)' can't be established.
ED25519 key fingerprint is SHA256:xK2V9B1mP3qR5tY7uI8oL0aS2dF4gH6jN8kM1nQ3rT5.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
warning: Permanently added '192.168.1.200' (ED25519) to the list of known hosts.
```

What is happening:

- The server sent its **host key** — a public key that identifies this specific server
- Your SSH client does not recognise it (you have never connected before)
- It asks you to verify and accept it
- Once you type `yes`, the fingerprint is saved in `~/.ssh/known_hosts`
- On all future connections, SSH checks that the fingerprint still matches

```
# View your known_hosts file
$ cat ~/.ssh/known_hosts
192.168.1.200 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKj...
github.com ssh-rsa AAAAB3NzaC1yc2EAAA...
```

The host key changed warning

If you ever see this, pay attention:

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!    @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
```

This means the server's fingerprint changed since you last connected. This happens legitimately when:

- The server was rebuilt or reinstalled
- You are connecting to a cloud VM that was terminated and a new one got the same IP

But it can also mean a man-in-the-middle attack is happening. Always investigate before proceeding.

```
# If you know the server was rebuilt, remove the old entry
$ ssh-keygen -R 192.168.1.200
# This deletes the old fingerprint from known_hosts
# On the next connection, you will be asked to accept the new one
```

Specifying which key to use

```
# -i specifies the identity file (private key)
$ ssh -i ~/.ssh/id_ed25519_prod sravan@prod.myapp.com

# Without -i, SSH tries all keys in ~/.ssh/ in order
# With -i, only that specific key is used
```

Section 6: The SSH Config File — Managing Many Servers Cleanly

Once you have more than two or three servers, typing full `ssh username@long.hostname.example.com -i ~/.ssh/id_ed25519_prod` every time is painful. The SSH config file lets you create shortcuts.

```
# The config file lives here
$ nano ~/.ssh/config

# Permissions must be correct
$ chmod 600 ~/.ssh/config
```

Config file syntax

```
Host <alias>
  HostName <real hostname or IP>
  User <username to log in as>
  IdentityFile <path to private key>
  Port <SSH port – default 22>
```

A real example

```
# Development server
Host devserver
  HostName 192.168.1.200
  User sravan
  IdentityFile ~/.ssh/id_ed25519

# Production web server
Host prod-web
  HostName ec2-54-123-45-67.compute-1.amazonaws.com
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519_prod
  Port 22

# Staging server
Host staging
  HostName staging.myapp.com
  User deploy
  IdentityFile ~/.ssh/id_ed25519

# Database server – only reachable via the prod-web server (jump host)
Host prod-db
  HostName 10.0.1.50
  User sravan
  IdentityFile ~/.ssh/id_ed25519_prod
  ProxyJump prod-web

# GitHub
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_github

# Match all remaining hosts – apply defaults
Host *
  ServerAliveInterval 60
  ServerAliveCountMax 3
  AddKeysToAgent yes
```

Now instead of:


```
$ ssh -i ~/.ssh/id_ed25519_prod ubuntu@ec2-54-123-45-67.compute-1.amazonaws.com
```

You type:

```
$ ssh prod-web
```

The SSH client reads the config, finds the `prod-web` block, and fills in all the details.

Config options explained

HostName	→ The real address (IP or DNS name) to connect to. Host is just your local alias – HostName is where SSH actually goes.
User	→ The username on the remote server. Without this, SSH uses your local username.
IdentityFile	→ Which private key to use for this host. Without this, SSH tries <code>~/.ssh/id_ed25519</code> , <code>id_rsa</code> , etc.
Port	→ The SSH port. Default is 22. Some servers use a non-standard port for security.
ProxyJump	→ Connect through another server first (jump host). Used when the target server has no public IP.
ServerAliveInterval	→ Send a keepalive packet every N seconds. Prevents the connection from dropping due to inactivity.
ServerAliveCountMax	→ How many keepalive failures before giving up.
AddKeysToAgent	→ Add the key to ssh-agent after use (avoids re-entering passphrase).

Section 7: Running Commands on a Remote Server Without an Interactive Shell

You do not always need a full interactive session. You can pass a command directly to SSH:

```
# Run a single command on the remote server and get the output back
$ ssh sravan@192.168.1.200 "hostname"
web01.myapp.com

# Check disk usage on the remote server
$ ssh sravan@192.168.1.200 "df -h"

# Check memory
$ ssh sravan@192.168.1.200 "free -h"

# Restart a service
$ ssh sravan@192.168.1.200 "sudo systemctl restart nginx"
```

```
# Run multiple commands – use && or ;  
$ ssh sravan@192.168.1.200 "cd /var/www/app && git pull && systemctl restart myapp"
```

This is enormously useful in deployment scripts. Instead of interactively logging in and typing commands, your script SSHes in, runs the command, and SSHes back out — all automatically.

```
# Example: deploy script using SSH  
#!/bin/bash  
SERVERS=("web01" "web02" "web03")  
APP_DIR="/var/www/myapp"  
  
for SERVER in "${SERVERS[@]"; do  
    echo "Deploying to $SERVER..."  
    ssh "$SERVER" "  
        cd $APP_DIR &&  
        git pull origin main &&  
        pip install -r requirements.txt --quiet &&  
        sudo systemctl restart myapp  
    " && echo "$SERVER: deploy OK" || echo "$SERVER: deploy FAILED"  
done
```

Running a local script on a remote server

```
# Pipe a local script through SSH – the script runs on the remote machine  
# but the file never needs to exist there  
$ ssh sravan@192.168.1.200 'bash -s' < local_script.sh  
  
# Or with arguments  
$ ssh sravan@192.168.1.200 'bash -s' < deploy.sh -- --env production --version v2.1
```

Section 8: Copying Files Over SSH

scp — Secure Copy

`scp` copies files over an SSH connection. It uses the same authentication as SSH — if you can `ssh` to a server, you can `scp` to it.

```
# Copy a local file TO a remote server  
$ scp localfile.txt sravan@192.168.1.200:/home/sravan/  
# Copies localfile.txt to /home/sravan/ on the remote server  
  
# Copy a file FROM a remote server to local  
$ scp sravan@192.168.1.200:/var/log/myapp/app.log ./  
# Copies app.log from the server to your current local directory  
  
# Copy a whole directory (recursive) – use -r  
$ scp -r ./myapp sravan@192.168.1.200:/var/www/
```

```
# Use a config alias
$ scp localfile.txt prod-web:/var/www/app/

# Specify a key
$ scp -i ~/.ssh/id_ed25519_prod config.json ubuntu@prod.myapp.com:/etc/myapp/
```

rsync — Smarter File Sync Over SSH

`rsync` is better than `scp` for anything beyond a single file copy. It:

- Only transfers files that changed (compares checksums)
- Preserves file permissions, timestamps, ownership
- Can resume interrupted transfers
- Shows progress

```
# Install rsync if needed
$ sudo apt install rsync -y

# Basic syntax: rsync [options] source destination
# A trailing slash on source means: copy the CONTENTS of the directory
# No trailing slash means: copy the directory itself

# Sync a local directory to a remote server
$ rsync -avz ./myapp/ sravan@192.168.1.200:/var/www/myapp/
# -a = archive mode (preserves permissions, timestamps, symlinks)
# -v = verbose (show what is being transferred)
# -z = compress data during transfer

# Sync from remote to local (backup)
$ rsync -avz sravan@192.168.1.200:/var/log/myapp/ ./logs/

# Dry run – show what WOULD be transferred without actually doing it
$ rsync -avz --dry-run ./myapp/ sravan@192.168.1.200:/var/www/myapp/

# Delete files on destination that no longer exist on source
$ rsync -avz --delete ./myapp/ sravan@192.168.1.200:/var/www/myapp/

# Use a specific SSH key
$ rsync -avz -e "ssh -i ~/.ssh/id_ed25519_prod" ./app/ ubuntu@prod.myapp.com:/var/www/app/

# Exclude certain files or directories
$ rsync -avz --exclude='.git' --exclude='node_modules' ./myapp/
sravan@192.168.1.200:/var/www/myapp/
```

Section 9: SSH Tunnelling — Accessing Services Through SSH

SSH tunnelling lets you securely access services on a remote server that are not exposed to the internet — like a database listening on `127.0.0.1` only.

Local port forwarding — the most common type

The scenario: Your database is on a server at `192.168.1.200`. PostgreSQL listens on `127.0.0.1:5432` on that server — it does not accept external connections for security. You want to connect to it from your laptop using a GUI tool like DBeaver or psql.

The solution: Create an SSH tunnel that forwards a local port on your machine to the remote port on the server.

```
# Local port forwarding
# -L local_port:remote_host:remote_port
$ ssh -L 5433:localhost:5432 sravan@192.168.1.200

# What this does:
# Opens a tunnel: your localhost:5433 → SSH → server's localhost:5432
# Any connection to your local port 5433 is forwarded through SSH to 5432 on the server
```

After running this, open a new terminal and:

```
# Connect to the tunnel – this actually connects to the remote PostgreSQL
$ psql -h localhost -p 5433 -U mydbuser -d mydbname
# Even though you are connecting to localhost:5433, the traffic goes through SSH to 5432 on the remote server
```

```
# Tunnel to a remote web app that only listens on localhost
$ ssh -L 8080:localhost:3000 sravan@192.168.1.200
# Now visit http://localhost:8080 in your browser
# You see the app running on port 3000 on the remote server

# Tunnel to a different machine through the SSH server
# Forward local 3306 → through SSH server → to another machine's MySQL
$ ssh -L 3306:db.internal.myapp.com:3306 sravan@192.168.1.200
# You can now access the internal database at localhost:3306
```

Run the tunnel in the background

```
# -N means do not run a command – just maintain the tunnel
# -f means go to background
$ ssh -N -f -L 5433:localhost:5432 sravan@192.168.1.200
# The tunnel runs in the background silently

# Find and kill it when done
$ ps aux | grep "ssh -N"
$ kill <PID>
```

Dynamic port forwarding — a SOCKS proxy

```
# Turn SSH into a full SOCKS proxy
$ ssh -D 1080 sravan@192.168.1.200
# Now configure your browser to use SOCKS5 proxy at localhost:1080
# All browser traffic goes through the SSH server
# Useful for accessing internal web services from a remote office network
```

Section 10: Debugging SSH — When It Does Not Work

SSH fails silently by default. `-v`, `-vv`, and `-vvv` add increasing levels of verbosity that tell you exactly where the handshake is breaking.

```
# Level 1 verbose – shows the connection and auth attempts
$ ssh -v sravan@192.168.1.200

# Level 2 – more detail about key exchange
$ ssh -vv sravan@192.168.1.200

# Level 3 – maximum detail, everything
$ ssh -vvv sravan@192.168.1.200
```

Reading the verbose output

```
$ ssh -v sravan@192.168.1.200

debug1: Connecting to 192.168.1.200 port 22.      ← Attempting TCP connection
debug1: Connection established.                  ← TCP connected
debug1: Server host key: ssh-ed25519 SHA256:...   ← Server sent its fingerprint
debug1: Host '192.168.1.200' is known and matches. ← Matches known_hosts – good
debug1: Offering public key: /home/sravan/.ssh/id_ed25519 ← Trying this key
debug1: Authentications that can continue: publickey ← Server accepts publickey auth
debug1: Server accepts key: /home/sravan/.ssh/id_ed25519 ← Key was accepted
debug1: Authentication succeeded (publickey).      ← Logged in
```

Common failures and fixes

Permission denied (publickey)

- The server did not accept any of your keys.
- Check: does your public key exist in `~/.ssh/authorized_keys` on the server?
- Check: are permissions correct? `chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys`
- Check: is the right key being offered? Use `ssh -v` to see which key is tried
- Check: is the server's `sshd` configured to allow key auth? (`PubkeyAuthentication yes` in `/etc/ssh/sshd_config`)

Connection refused

- Nothing is listening on port 22 on that IP
- Check: is `sshd` running? `systemctl status ssh`

- Check: is the IP correct?
- Check: is a firewall blocking port 22? `ss -tulnp | grep :22`

Connection timed out

- The packets are being dropped (firewall), not rejected
- A cloud security group may be blocking port 22
- The server may be powered off

Host key verification failed

- The server's fingerprint changed since your last connection
- If the server was rebuilt: `ssh-keygen -R <hostname>`
- If unexpected: investigate before connecting

Warning: Unprotected private key file

- Your private key has too-loose permissions
- Fix: `chmod 600 ~/.ssh/id_ed25519`

Section 11: ssh-agent — Avoid Typing Your Passphrase Repeatedly

If your private key has a passphrase (it should in production), you would have to type it every time you use the key. `ssh-agent` solves this by holding your unlocked key in memory for the session.

```
# Start ssh-agent (usually already running in a desktop session)
$ eval $(ssh-agent)
Agent pid 1234

# Add your key – type the passphrase once
$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/sravan/.ssh/id_ed25519:
Identity added: /home/sravan/.ssh/id_ed25519 (sravan-devops-training)

# List keys currently loaded in the agent
$ ssh-add -l
256 SHA256:xK2V9B1mP3qR5tY7 sravan-devops-training (ED25519)

# Now SSH uses the key without asking for the passphrase again (until you log out)
$ ssh prod-web      # no passphrase prompt
$ ssh staging        # no passphrase prompt
```

Section 12: Hands-On Practice for Day 18

Exercise 1 — Generate and inspect keys (15 minutes)

```
# Generate your main key if you have not already
$ ssh-keygen -t ed25519 -C "sravan-devops-training"

# Check what was created
```

```
$ ls -la ~/.ssh/

# Read your public key
$ cat ~/.ssh/id_ed25519.pub

# Read your private key – note the header/footer
$ cat ~/.ssh/id_ed25519

# Verify the permissions are correct
$ stat ~/.ssh/id_ed25519
# Look for "0600" or "Access: (0600/-rw-----)"

# Generate a second key for practice
$ ssh-keygen -t ed25519 -C "sravan-github-key" -f ~/.ssh/id_ed25519_github
# Press Enter to skip passphrase

# List all keys now
$ ls -la ~/.ssh/
```

Exercise 2 — Set up the SSH config file (15 minutes)

```
# Create the config file
$ nano ~/.ssh/config
```

Add this content (adjust the IP for a server you actually have, or use localhost):

```
# Local machine shortcut (loop back to yourself – for testing config syntax)
Host local
    HostName 127.0.0.1
    User YOUR_USERNAME
    IdentityFile ~/.ssh/id_ed25519
    Port 22

# GitHub
Host github.com
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_github

# Global defaults
Host *
    ServerAliveInterval 60
    ServerAliveCountMax 3
```

```
# Set correct permissions
$ chmod 600 ~/.ssh/config

# Verify the config is valid – ssh -G shows resolved config for an alias
$ ssh -G local
# Should output all the resolved values for the 'local' host alias
```

Exercise 3 — SSH to localhost (10 minutes)

If you have sshd installed locally, you can SSH to yourself for practice:

```
# Install and start SSH server
$ sudo apt install openssh-server -y
$ sudo systemctl start ssh
$ sudo systemctl status ssh

# Copy your own public key to your own authorized_keys
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub $(whoami)@localhost
# OR manually:
$ cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
$ chmod 600 ~/.ssh/authorized_keys

# Now SSH to yourself
$ ssh $(whoami)@localhost
# You should get a new shell without entering a password

# Run a command without opening a shell
$ ssh localhost "hostname && whoami && date"

# Exit
$ exit
```

Exercise 4 — Run remote commands in a loop (15 minutes)

```
# Create a list of "servers" to test against (all localhost for now)
# In real work, these would be your actual server hostnames

$ nano remote_check.sh
```

```
#!/bin/bash
# Simulates running a command on multiple servers
# Uses localhost as a stand-in for real servers

set -uo pipefail

SERVERS=("localhost" "localhost" "localhost")
NAMES=("web01" "web02" "web03")
PASS=0
FAIL=0

echo "Running remote health check..."
echo "===== "

for i in "${!SERVERS[@]}"; do
    SERVER="${SERVERS[$i]}"
    NAME="${NAMES[$i]}"

    # Run a command on the remote server
```



```

RESULT=$(ssh -o ConnectTimeout=3 -o BatchMode=yes \
"$SERVER" "echo OK && hostname && uptime" 2>/dev/null)

if [ $? -eq 0 ]; then
    echo "    [PASS] $NAME - $RESULT"
    PASS=$((PASS + 1))
else
    echo "    [FAIL] $NAME - could not connect"
    FAIL=$((FAIL + 1))
fi
done

echo "===== "
echo "Results: $PASS passed, $FAIL failed"

```

```

$ chmod +x remote_check.sh
$ ./remote_check.sh

```

`-o BatchMode=yes` tells SSH not to prompt for anything — if it cannot authenticate silently, it fails immediately. This is essential in scripts because a hanging password prompt will freeze your automation.

Exercise 5 — SCP file transfer (10 minutes)

```

# Create a test file
$ echo "Hello from $(hostname) on $(date)" > /tmp/testfile.txt

# Copy it to localhost (simulating a server)
$ scp /tmp/testfile.txt $(whoami)@localhost:/tmp/testfile_copy.txt

# Verify it arrived
$ ssh localhost "cat /tmp/testfile_copy.txt"

# Copy it back with a different name
$ scp $(whoami)@localhost:/tmp/testfile_copy.txt /tmp/returned.txt
$ cat /tmp/returned.txt

# Copy a directory
$ mkdir -p /tmp/testdir
$ echo "file1" > /tmp/testdir/a.txt
$ echo "file2" > /tmp/testdir/b.txt
$ scp -r /tmp/testdir $(whoami)@localhost:/tmp/testdir_remote/

# Verify
$ ssh localhost "ls /tmp/testdir_remote/"

```

Exercise 6 — SSH tunnel (15 minutes)

This exercise sets up a local port forward to experience tunnelling:

```

# Start a simple HTTP server on the remote side (localhost port 8888)
# Terminal 1 – simulate a "remote" service

```

```
$ python3 -m http.server 8888 &
SERVER_PID=$!
echo "Server PID: $SERVER_PID"

# Terminal 2 – create a tunnel
# Forward your local port 9999 to localhost:8888 through SSH
$ ssh -N -f -L 9999:localhost:8888 $(whoami)@localhost

# Terminal 3 – connect through the tunnel
$ curl http://localhost:9999
# You see the directory listing from python's HTTP server
# Traffic went: your curl → local 9999 → SSH tunnel → 8888

# When done – kill the tunnel and the server
$ kill $(pgrep -f "ssh -N.*9999")
$ kill $SERVER_PID
```

What's Next — Day 19

Tomorrow is Vim. You cannot avoid it. Every Linux server has Vim (or vi). When you SSH into a server to fix a config file and there is no nano, no VSCode, no GUI — Vim is what you get. You do not need to love it. You need to be able to open a file, make a change, and save without panicking.

```
# Start Vim's built-in tutorial right now – it takes 30 minutes
# and teaches you everything you need for the next few months
$ vimtutor

# If you want a preview – open Vim and try to exit it
# (this is famously confusing for beginners)
$ vim /etc/hosts

# Now try to exit without saving... figure it out.
# Hint: the answer involves pressing Escape and then typing something starting with :
```